# Robot Actuator Loop

Single-threaded UDP control loop · 9 actuators · 1kHz · p99 < 200µs

| 10.2M | 5 × 5 min | 94.4µs |
| --- | --- | --- |
| Total Measurements | Benchmark Runs | Mean p99 (corrected) |

Ashley Zhou

# System Overview

## What It Is

**Two C programs over UDP on localhost:**

**actuator.c:** simulates one robot joint. Listens on its own UDP port, replies with a sine-wave position (180·sin(2πt)).

**orchestrator.c:** single-threaded control loop. Sends commands to all 9 actuators every 1ms (fast tick). Every 1 second, appends the last 1000 commands to JSON on disk (slow tick).

**Message format:** 20-byte struct: actuator_id, counter, timestamp_ns, position.

## The Engineering Challenge

**Single-thread constraint:**
No threads, no subprocesses. Fast tick and slow tick both run on one OS thread.

**Latency deadline:**
p99 RTT must stay under 200µs even while disk I/O periodically occurs.

**Bus architecture:**
3 buses × 3 actuators = 9 total. Each gets a unique port (PORT_BASE + bus×3 + id) to avoid routing ambiguity.

# Timing Methodology

## RTT Definition

send_times[i] = now_ns() stamped just before sendto(). RTT recorded when recvfrom() returns the response. Uses mach_absolute_time() on Mac for nanosecond-precision monotonic clock.

## Non-Blocking Sockets

fcntl(sock, F_SETFL, O_NONBLOCK) on each of 9 sockets. recvfrom() returns instantly with or without data. Poll loop spins over all 9 sockets until all respond or the 0.9ms deadline passes.

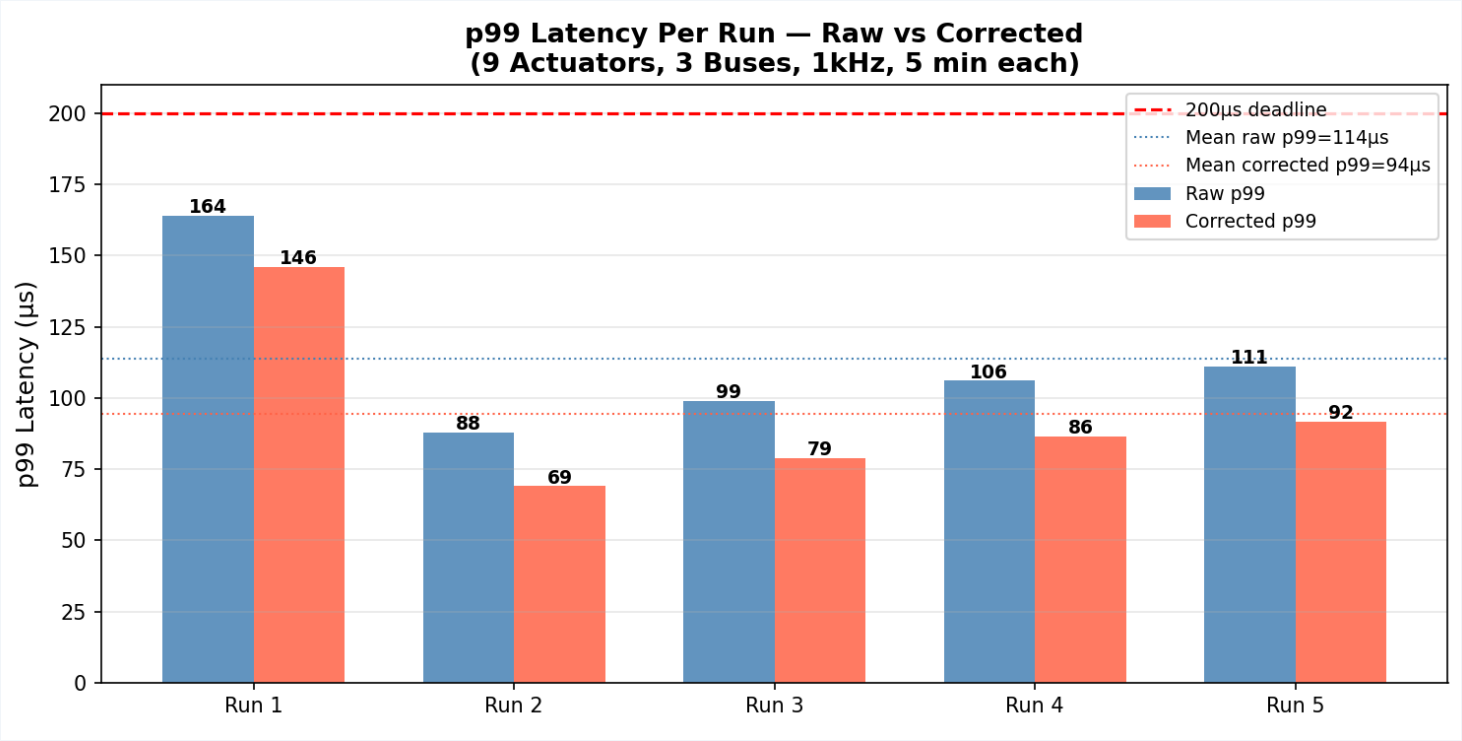
## Benchmark Setup

5 independent runs × 5 minutes = 10,166,040 total measurements. Each row records: counter, bus, target_id, target_idx (send order 0-8), rtt_us.

## Send-Offset Correction

Sequential sends create a gradient in raw RTTs (resulting in additional time for earlier targets). Correction: overhead = (avg_idx0 − avg_idx8) / 8. corrected = raw − ((8 − idx) × overhead).

# Results - p99 Latency per Run



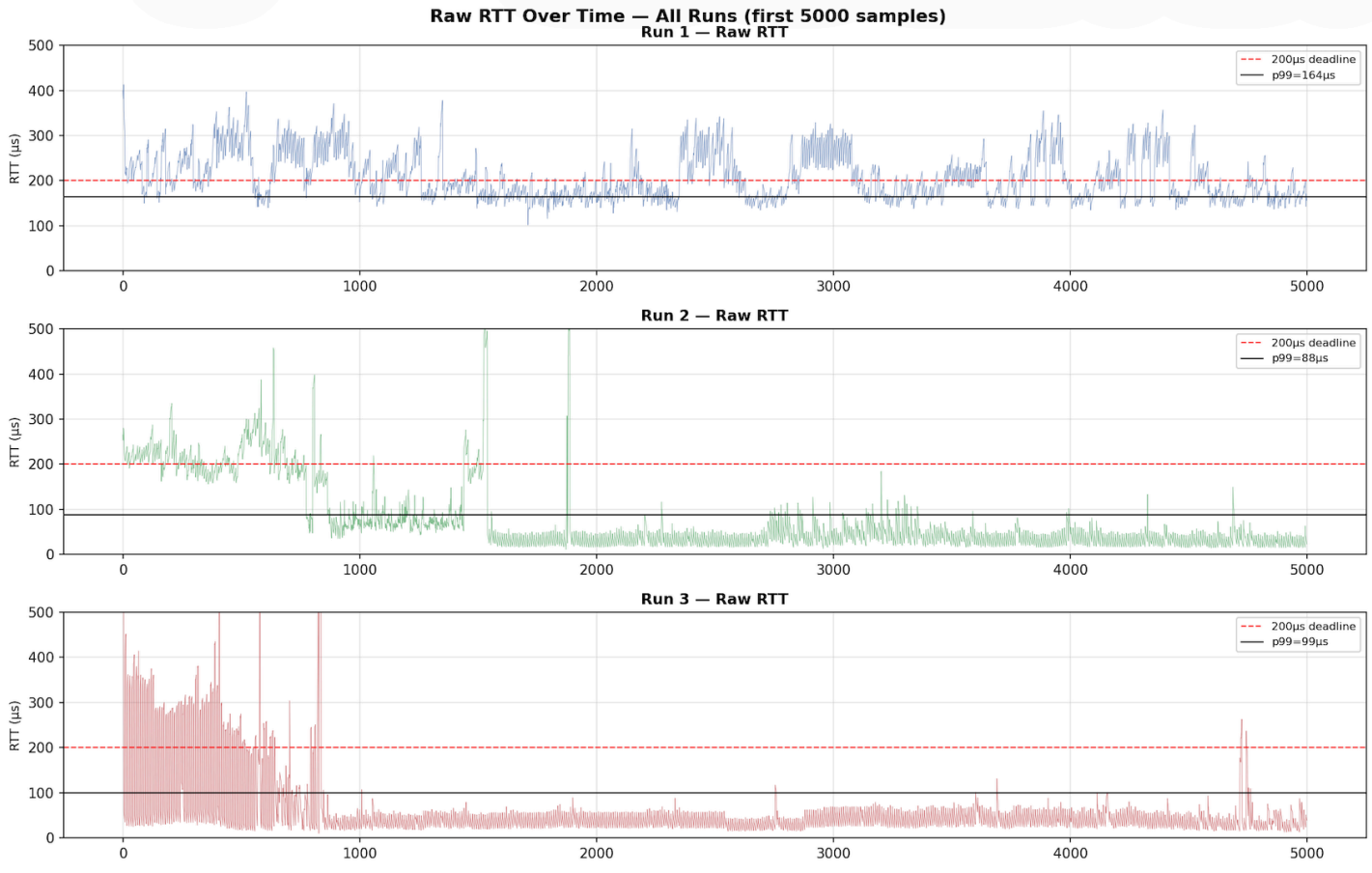

**113.6µs**
Mean raw p99

**94.4µs**
Mean corrected p99

**< 200µs**
All 5 runs pass deadline

*Run 1 outlier (164µs p99, 390ms max): background OS interference / waking from sleep. All 5 runs meet 200µs deadline.*

# Results - RTT Over Time (Runs 1-3)

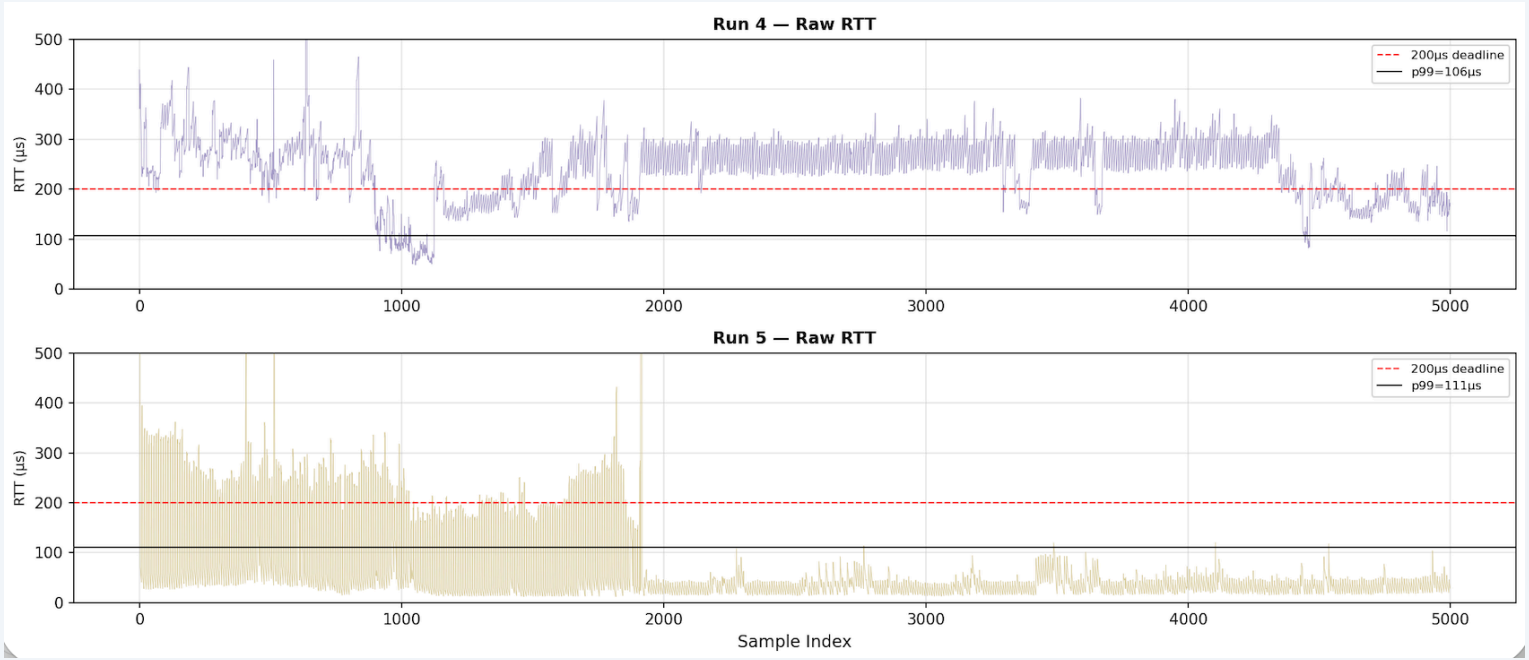



Runs 2, 3, 5: warm-up drop ~200µs → 50-100µs. Cache warming, CPU freq scaling, page faults.

Run 4: multiple transitions - intermittent background process interference.

Run 1: no warm-up drop - elevated throughout, consistent with 390ms outlier.

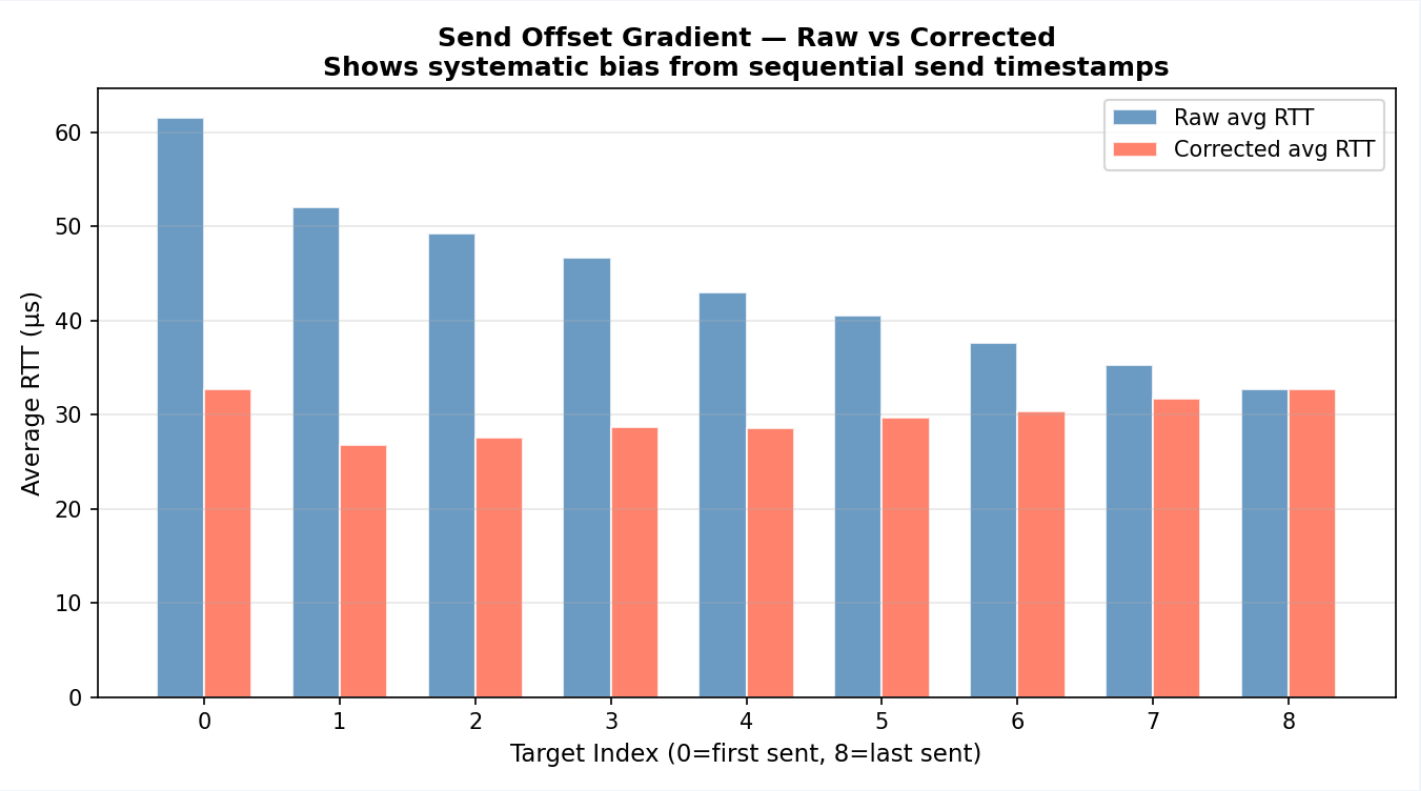
# Results - RTT Over Time (Runs 4-5)




Runs 2, 3, 5: warm-up drop ~200µs → 50-100µs. Cache warming, CPU freq scaling, page faults. (Runs 2 and 3 shown on previous slide).

Run 4: multiple transitions - intermittent background process interference.

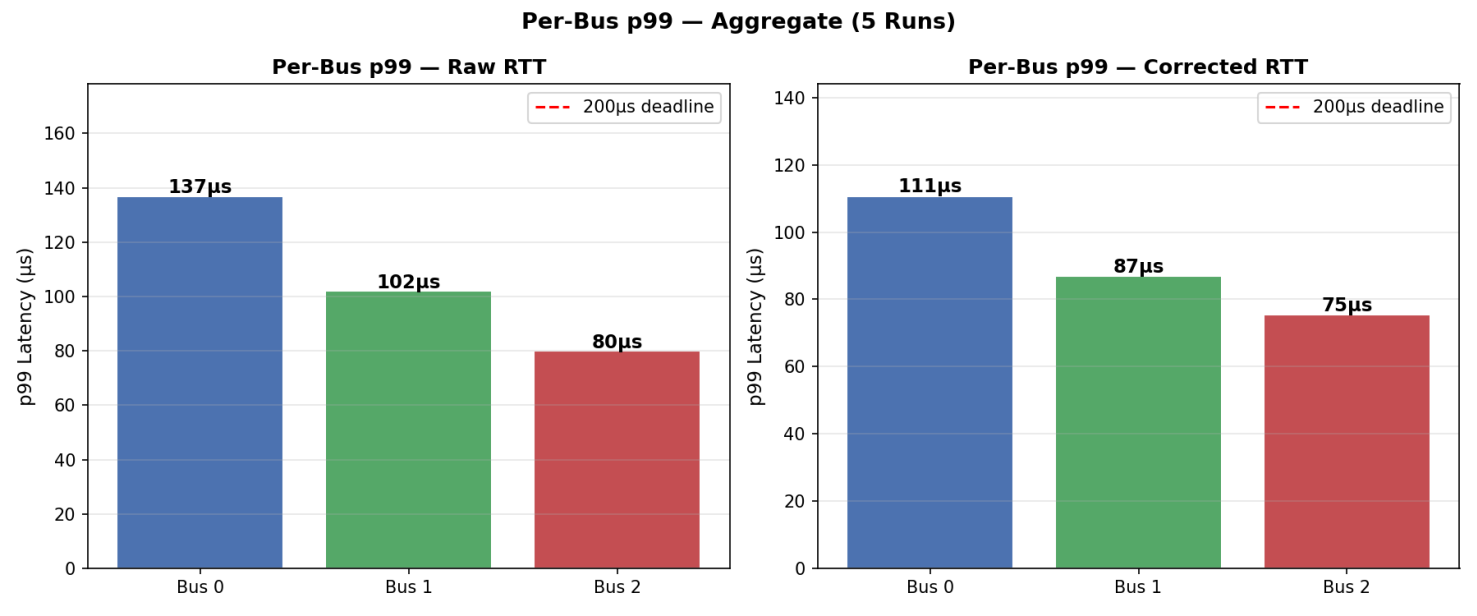
# Results - Send-Offset Gradient




Send-Offset Gradient

Raw RTT slopes from ~62µs (idx 0, sent first) to ~33µs (idx 8, sent last).

# Results - Per-Bus p99




Per-Bus p99

Bus 0 (targets 0-2, sent and polled first) shows highest p99 even after correction (send-order bias dominates per-bus tail behavior).

# Key Takeaways

## 01

### Blocking → Non-Blocking was the critical change

p99 dropped from 265µs (failing) to 94-114µs (passing). Blocking recvfrom() froze the thread, while non-blocking polling reads whichever responses are ready.

## 02

### Single-thread design holds up by sequencing correctly

RTTs are recorded before write_log() runs. By construction, the slow tick cannot inflate that tick's measurement.

## 03

### OS jitter dominates the tail, not the code

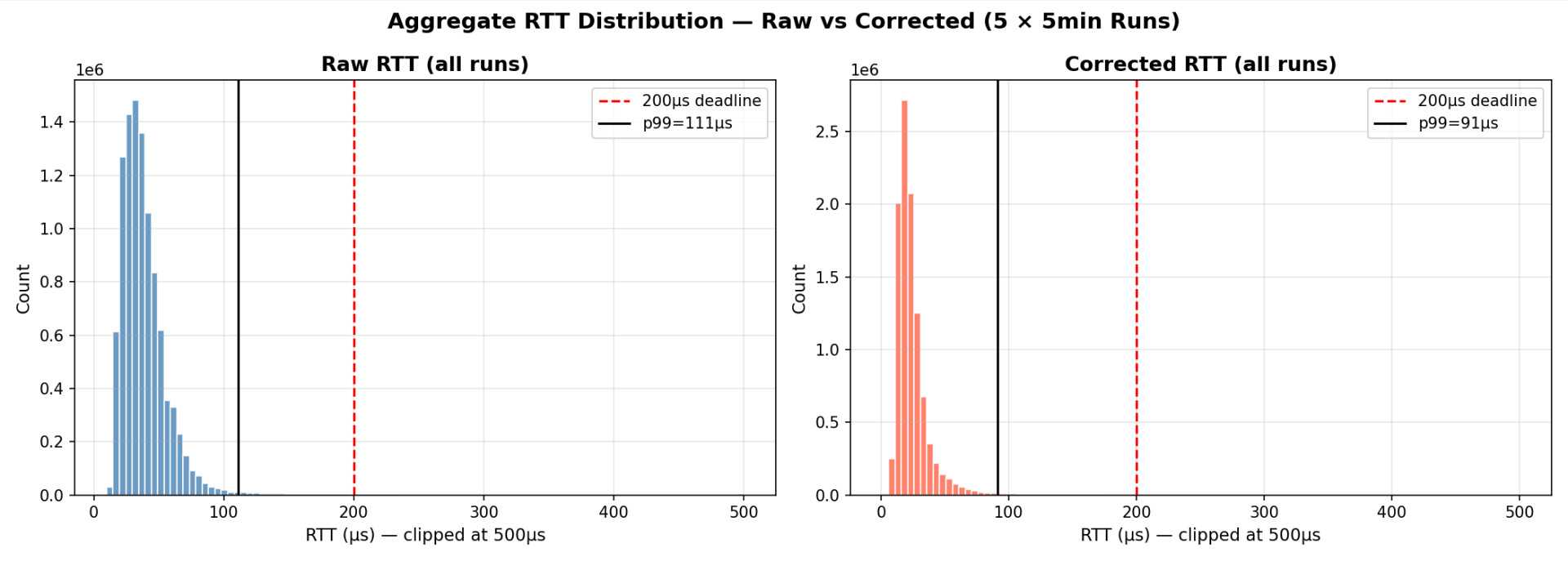
Tight cluster (10-50µs) = true round-trip speed. The tail and 390ms outlier are OS scheduling events, which is expected on a general-purpose desktop OS.

## 04

### Next Steps: Using SO_TIMESTAMP to remove loop overhead

SO_TIMESTAMP captures kernel arrival time, removing receive-loop read-order bias, eliminating loop overhead from being included when measuring RTT.

# Results - Latency Distribution




**Tight cluster (10-50µs):** true UDP round-trip + sine computation (system working as intended). **Long right tail:** rare OS scheduler preemptions. Corrected histogram (right) is tighter (send-loop bias removed).